
Curso de especialización en Desarrollo de aplicaciones en lenguaje Python

UT4.- Acceso a bases de datos con Python

Tarea 1

Una biblioteca desea un programa sencillo para **gestionar su catálogo** y el **préstamo/devolución** de libros.

El personal usará una aplicación en **consola** con un menú.

La información debe **persistirse** en una base de datos **SQLite (biblioteca.db)** para que, al cerrar y volver a abrir el programa, los libros sigan estando disponibles.

El proyecto debe incluir:

- **Biblioteca.py:**
 - clase **Libro:**
 - contendrá los siguientes **atributos privados:**
 - **título:** título del libro.
 - **autor:** autor del libro.
 - **isbn:** isbn del libro, **tiene que ser único.**
 - **disponible:** indicará si el libro está disponible o no para préstamo.
 - **clase Biblioteca** que almacenará una lista de libros:
- **main.py:** menú interactivo en consola.

```
*****
Gestión de la Biblioteca
*****
1. Mostrar libros
2. Agregar libro
3. Buscar un libro
4. Eliminar libro
5. Prestar libro
6. Devolver libro
7. Salir
Seleccione una opción:
```

- **Conexion_BD.py:** gestión de conexión y operaciones SQL.
- **biblioteca.db:** base de datos SQLite.

Gestión de libros (CRUD)

El sistema debe permitir:

1. **Mostrar libros**
 - Listar todos los libros disponibles en la biblioteca.
 - Mostrar: *título, autor, ISBN y estado* (Disponible / Prestado).
2. **Agregar libro**
 - Pedir por consola: **título, autor, isbn**. El isbn debe ser único
 - Validar datos.
 - Insertar en memoria (lista de libros) y persistir en BD.
3. **Buscar libro**
 - Busca un libro por su **ISBN**
 - Si no existe, informar al usuario sin interrumpir el programa.
4. **Eliminar libro**
 - Eliminar un libro por **ISBN** y borrar en BD.
 - Si no existe, informar al usuario sin interrumpir el programa.
5. **Prestar libro**
 - Prestar por **ISBN**.
 - Si está disponible, cambiar estado a “prestado” y persistir.
 - Si ya está prestado, informar: “Libro no disponible”.
6. **Devolver libro**
 - Devolver por **ISBN**.
 - Si estaba prestado, cambiar estado a “disponible” y persistir.
 - Si ya estaba disponible, informar: “Libro no prestado”.

Reglas de duplicidad

Debes definir claramente y aplicar una regla de identificación. Como mínimo:

- **ISBN** debe ser **único**.
- Si se intenta registrar un libro con un ISBN ya existente:
 - No se inserta.
 - Se informa al usuario.

Estado del libro

- Un libro tiene un estado: **disponible** (True/1) o **prestado** (False/0).

Tabla y esquema mínimo

La tabla debe llamarse **libros** y contener al menos:

- **id** (autoincremental, PK)
- **titulo** (TEXT, NOT NULL)
- **autor** (TEXT, NOT NULL)
- **isbn** (TEXT, NOT NULL)
- **disponible** (INTEGER, NOT NULL) → 1 disponible / 0 prestado

Sincronización memoria ↔ BD

- Al iniciar el programa, la biblioteca debe **cargar** los libros desde la BD a la lista interna.
- Cada operación (alta/baja/préstamo/devolución) debe reflejarse:
 - en la **lista** en memoria
 - en la **base de datos**

Validaciones obligatorias

- **titulo** y **autor**: deben ser **str** no vacíos.
- **isbn**: debe ser **str** no vacío.

Manejo de errores

- El programa **no debe cerrarse** por errores esperables (libro no encontrado, ISBN duplicado, etc.).
- Las excepciones deben gestionarse:
 - donde corresponda (lógica o interfaz)
 - con mensajes claros para el usuario

Entregables

El alumnado debe entregar:

1. Código fuente completo funcionando.
2. **README.md** con:
 - descripción del proyecto
 - cómo ejecutarlo
 - decisiones de diseño (p. ej., criterio de duplicidad ISBN vs título)
3. Un pequeño conjunto de **datos de prueba** o instrucciones para generarlos.